Integración de datos

Tarea Práctica - Clase 5

Sergio Enrique Morel Peralta
Facultad Politécnica - UNA, 03.04.2026

1. Tarea 5: Transformación de Datos con dbt

Esta tarea implementa un proyecto **dbt (data build tool)** completo sobre los datos cargados con Airbyte en clases anteriores: pronósticos meteorológicos de **OpenWeather** (`weather.weather`) y metadatos del repositorio **SEMP/lib-utilidades** en GitHub (`github.branches` y `github.stargazers`), disponibles en MotherDuck bajo la base de datos `md:airbyte_curso`. El proyecto organiza las transformaciones en tres capas — **staging**, **intermediate** y **marts** — siguiendo las convenciones de dbt.

1.1. Datos fuente

Los datos crudos sincronizados por Airbyte en clases anteriores son:

Tabla	Filas	Descripción
`weather.weather`	40	Pronóstico de 5 días con intervalos de 3 horas — OpenWeather API
`github.branches`	1	Rama principal del repositorio SEMP/lib-utilidades
`github.stargazers`	1	Estrella dada al repositorio SEMP/lib-utilidades

1.2. Configuración del proyecto dbt

1.2.1. Instalación y entorno virtual

El proyecto utiliza **dbt-core** con el adaptador **dbt-duckdb**, que permite conectarse a MotherDuck (DuckDB en la nube). Se creó un entorno virtual Python en `workspaces/dbt-duckdb/.venv` e instalaron las dependencias declaradas en `requirements.txt`:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r mi_proyecto_dbt/requirements.txt
```

1.2.2. Estructura del proyecto

El proyecto dbt se ubica en `workspaces/dbt-duckdb/mi_proyecto_dbt/` e incluye los archivos de configuración y los modelos organizados en tres capas:

```
mi_proyecto_dbt/
├── dbt_project.yml          # Configuración principal del proyecto
├── profiles.yml             # Conexión a MotherDuck
├── models/
│   ├── staging/
│   │   ├── _sources.yml     # Declaración de fuentes crudas
│   │   ├── stg_weather__forecast.sql
│   │   ├── stg_github__stargazers.sql
│   │   └── stg_github__branches.sql
│   ├── intermediate/
│   │   └── int_github_actividad.sql
│   └── marts/
│       ├── obt_pronostico.sql
│       └── obt_github_actividad.sql
└── requirements.txt
```

1.2.3. Conexión a MotherDuck (`profiles.yml`)

La conexión se configura en `profiles.yml` dentro del proyecto. El token de acceso se lee desde la variable de entorno `MOTHERDUCK_TOKEN` para evitar exponer credenciales en el repositorio:

```
mi_proyecto_dbt:
  outputs:
    dev:
      type: duckdb
      path: "md:airbyte_curso?
motherduck_token={{env_var('MOTHERDUCK_TOKEN')}}"
  target: dev
```

1.2.4. Verificación de la conexión (`dbt debug`)

Se verificó la conexión a MotherDuck ejecutando `dbt debug`, obteniendo el resultado:

```
Running with dbt=1.11.7
adapter type: duckdb
adapter version: 1.10.1

profiles.yml file    [OK found and valid]
dbt_project.yml file [OK found and valid]
git                  [OK found]
Connection test:     [OK connection ok]

All checks passed!
```

1.3. Modelos staging

La capa de staging limpia y renombra los campos de las tablas crudas sin aplicar lógica de negocio. Cada modelo sigue el patrón `stg_<source>__<tabla>.sql` y usa dos CTEs: `source` (referencia a la tabla cruda vía `{{ source() }}`) y `renamed` (selección y renombrado de columnas).

Un detalle importante: Airbyte almacenó los campos anidados del JSON como **structs** de DuckDB en lugar de aplanarlos con guiones bajos. Por ello fue necesario usar notación de punto (`sys.pod`, `main.temp`, `wind.speed`) y notación de corchetes para claves con nombres reservados o que comienzan con número (`clouds['all']`, `rain['3h']`). Los elementos del array `weather` se acceden con índice 1-based (`weather[1].id`).

1.3.1. `stg_weather__forecast`

Limpia la tabla `weather.weather` (40 filas). Extrae los campos de fecha/hora desde `dt_txt`, fija las coordenadas de ubicación (fuente tiene una sola ciudad), y desanida los structs `sys`, `main`, `wind`, `clouds`, `rain` y el array `weather`.

```
WITH source AS
(
    SELECT *
    FROM {{ source('weather', 'weather') }}
),

renamed AS
(
    SELECT
        -- tiempo
        dt                              AS dt_unix,
        dt_txt,
        CAST(dt_txt AS DATE)            AS fecha,
```

```
        HOUR(CAST(dt_txt AS TIMESTAMP)) AS hora,
        YEAR(CAST(dt_txt AS TIMESTAMP)) AS anio,
        MONTH(CAST(dt_txt AS TIMESTAMP)) AS mes,
        DAY(CAST(dt_txt AS TIMESTAMP))  AS dia,
        sys.pod                         AS parte_dia,

        -- ubicacion (fija para esta fuente)
        -25.5309750                     AS latitud,
        -54.6388360                     AS longitud,
        'Ciudad del Este'               AS ciudad,
        'PY'                            AS pais,

        -- condicion climatica
        weather[1].id                   AS condicion_codigo,
        weather[1].main                 AS condicion_principal,
        weather[1].description          AS condicion_descripcion,
        weather[1].icon                 AS condicion_icono,

        -- temperatura
        main.temp                       AS temperatura_c,
        main.feels_like                 AS sensacion_termica_c,
        main.temp_min                   AS temp_min_c,
        main.temp_max                   AS temp_max_c,

        -- humedad y presion
        main.humidity                   AS humedad_pct,
        main.pressure                   AS presion_hpa,
        main.sea_level                  AS presion_mar_hpa,
        main.grnd_level                 AS presion_suelo_hpa,

        -- viento
        wind.speed                      AS velocidad_viento_ms,
        wind.deg                        AS dir_viento_deg,
        wind.gust                       AS rafaga_viento_ms,

        -- nubes y precipitacion
        clouds['all']                   AS cobertura_nubes_pct,
        pop                             AS prob_precipitacion,
        rain['3h']                      AS lluvia_3h_mm,
        visibility                      AS visibilidad_m

    FROM source
)

SELECT * FROM renamed
```

1.3.2. stg_github__stargazers

Limpia la tabla `github.stargazers` (1 fila). El campo `user` es un struct; `user_id` se encuentra en el nivel raíz de la tabla.

```
WITH source AS
(
    SELECT *
    FROM {{ source('github', 'stargazers') }}
),

renamed AS
(
    SELECT
        -- usuario
        user_id                         AS usuario_github_id,
        user.login                      AS usuario_login,
        user.type                       AS usuario_tipo,
        user.site_admin                 AS usuario_es_admin,
        user.html_url                   AS usuario_perfil_url,
        user.avatar_url                 AS usuario_avatar_url,

        -- repositorio
        repository                      AS repositorio_nombre_completo,

        -- evento
        starred_at,
        CAST(starred_at AS DATE)        AS fecha,
        YEAR(starred_at)                AS anio,
        MONTH(starred_at)               AS mes,
        DAY(starred_at)                 AS dia

    FROM source
)

SELECT * FROM renamed
```

1.3.3. stg_github__branches

Limpia la tabla `github.branches` (1 fila). El campo `commit` es un struct con `sha` y `url`.

```
WITH source AS
(
    SELECT *
    FROM {{ source('github', 'branches') }}
),

renamed AS
(
    SELECT
```

```
        repository                      AS repositorio_nombre_completo,
        name                            AS rama_nombre,
        commit.sha                      AS rama_commit_sha,
        commit.url                      AS rama_commit_url,
        protected                       AS rama_protegida

    FROM source
)

SELECT * FROM renamed
```

1.4. Modelo intermediate

La capa intermediate aplica lógica de negocio y combina modelos de staging. El modelo `int_github_actividad` une `stg_github__stargazers` con `stg_github__branches` mediante un `LEFT JOIN` sobre `repositorio_nombre_completo`, y deriva las columnas `repositorio_propietario` y `repositorio_nombre` partiendo el nombre completo por `/`.

```
WITH stargazers AS
(
    SELECT *
    FROM {{ ref('stg_github__stargazers') }}
),

branches AS
(
    SELECT *
    FROM {{ ref('stg_github__branches') }}
),

joined AS
(
    SELECT
        -- evento estrella
        stargazers_t.starred_at,
        stargazers_t.fecha,
        stargazers_t.anio,
        stargazers_t.mes,
        stargazers_t.dia,

        -- usuario
        stargazers_t.usuario_github_id,
        stargazers_t.usuario_login,
        stargazers_t.usuario_tipo,
        stargazers_t.usuario_es_admin,
        stargazers_t.usuario_perfil_url,
        stargazers_t.usuario_avatar_url,
```

```
        -- repositorio
        stargazers_t.repositorio_nombre_completo,
        SPLIT_PART(stargazers_t.repositorio_nombre_completo, '/', 1) AS
repositorio_propietario,
        SPLIT_PART(stargazers_t.repositorio_nombre_completo, '/', 2) AS
repositorio_nombre,

        -- rama principal
        branches_t.rama_nombre            AS rama_principal_nombre,
        branches_t.rama_commit_sha        AS rama_principal_sha,
        branches_t.rama_protegida         AS rama_principal_protegida

    FROM stargazers AS stargazers_t
    LEFT JOIN branches AS branches_t
        ON stargazers_t.repositorio_nombre_completo =
branches_t.repositorio_nombre_completo
)

SELECT * FROM joined
```

1.5. Modelos mart

La capa mart expone las tablas finales listas para consumo analítico. Se eligió el modelo **OBT (One Big Table)** dado el bajo volumen de datos (40 filas para weather, 1 para GitHub) y la ausencia de redundancia real a esa escala. Ambos modelos se materializan como `table`.

1.5.1. `obt_pronostico`

Tabla única de 32 columnas que aplana todos los atributos del pronóstico meteorológico. Consume directamente `stg_weather__forecast` y agrega `pronostico_id` como clave primaria vía `ROW_NUMBER()`.

```
{{ config(materialized='table') }}

WITH forecast AS
(
    SELECT *
    FROM {{ ref('stg_weather__forecast') }}
),

final AS
(
    SELECT
        ROW_NUMBER() OVER () AS pronostico_id,
        dt_unix,
        dt_txt,
        fecha,
        hora,
```

```
        anio,
        mes,
        dia,
        parte_dia,
        latitud,
        longitud,
        ciudad,
        pais,
        condicion_codigo,
        condicion_principal,
        condicion_descripcion,
        condicion_icono,
        temperatura_c,
        sensacion_termica_c,
        temp_min_c,
        temp_max_c,
        humedad_pct,
        presion_hpa,
        presion_mar_hpa,
        presion_suelo_hpa,
        visibilidad_m,
        velocidad_viento_ms,
        dir_viento_deg,
        rafaga_viento_ms,
        cobertura_nubes_pct,
        prob_precipitacion,
        lluvia_3h_mm
    FROM forecast
)

SELECT * FROM final
```

1.5.2. obt_github_actividad

Tabla única de 18 columnas que combina los datos de la estrella, el usuario y la rama principal del repositorio. Consume `int_github_actividad` y agrega `estrella_id` como clave primaria.

```
{{ config(materialized='table') }}

WITH actividad AS
(
    SELECT *
    FROM {{ ref('int_github_actividad') }}
),

final AS
(
    SELECT
```

```
        ROW_NUMBER() OVER () AS estrella_id,
        starred_at,
        fecha,
        anio,
        mes,
        dia,
        usuario_login,
        usuario_github_id,
        usuario_tipo,
        usuario_es_admin,
        usuario_perfil_url,
        usuario_avatar_url,
        repositorio_nombre_completo,
        repositorio_propietario,
        repositorio_nombre,
        rama_principal_nombre,
        rama_principal_sha,
        rama_principal_protegida
    FROM actividad
)

SELECT * FROM final
```

1.6. DAG del proyecto

El DAG (Directed Acyclic Graph) de dbt representa las dependencias entre modelos. Se generó ejecutando `dbt docs generate` y se visualizó con `dbt docs serve`. El proyecto contiene dos pipelines independientes, capturados por separado.

Pipeline weather: la fuente cruda `weather.weather` fluye hacia `stg_weather__forecast` (staging) y directamente a `obt_pronostico` (mart), sin capa intermediate dado que no hay joins con otras fuentes.

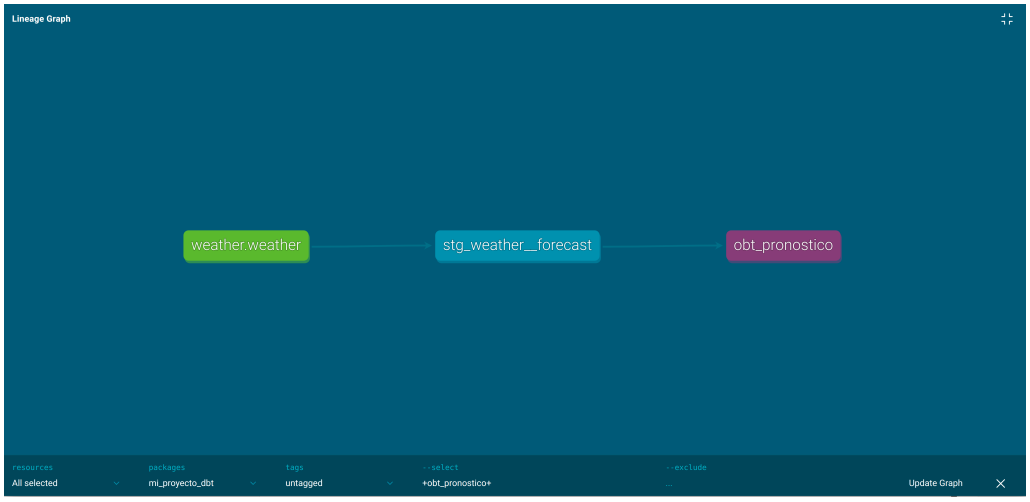

Figura 1: DAG — pipeline weather (`+obt_pronostico+`)

Pipeline GitHub: las fuentes `github.branches` y `github.stargazers` fluyen hacia sus modelos de staging, se combinan en `int_github_actividad` (intermediate) y producen `obt_github_actividad` (mart).

Figura 2: DAG — pipeline GitHub (`+int_github_actividad+`)